Aula 02: Microsserviços na Prática com Flask

Para futuras consultas aos comandos, o PDF está disponível para download.

Todo o material e arquivos desenvolvidos durante esta aula podem ser encontrados no seguinte repositório do GitHub:

https://github.com/kelvinalmeida/Aula-02-Microservicos-na-Pratica-com-Flask

Para baixar o Python acesse o site oficial:

https://www.python.org/

Irei Utilizar Flask

Nos dois exemplos a seguir irei utilizar Flask, porque:

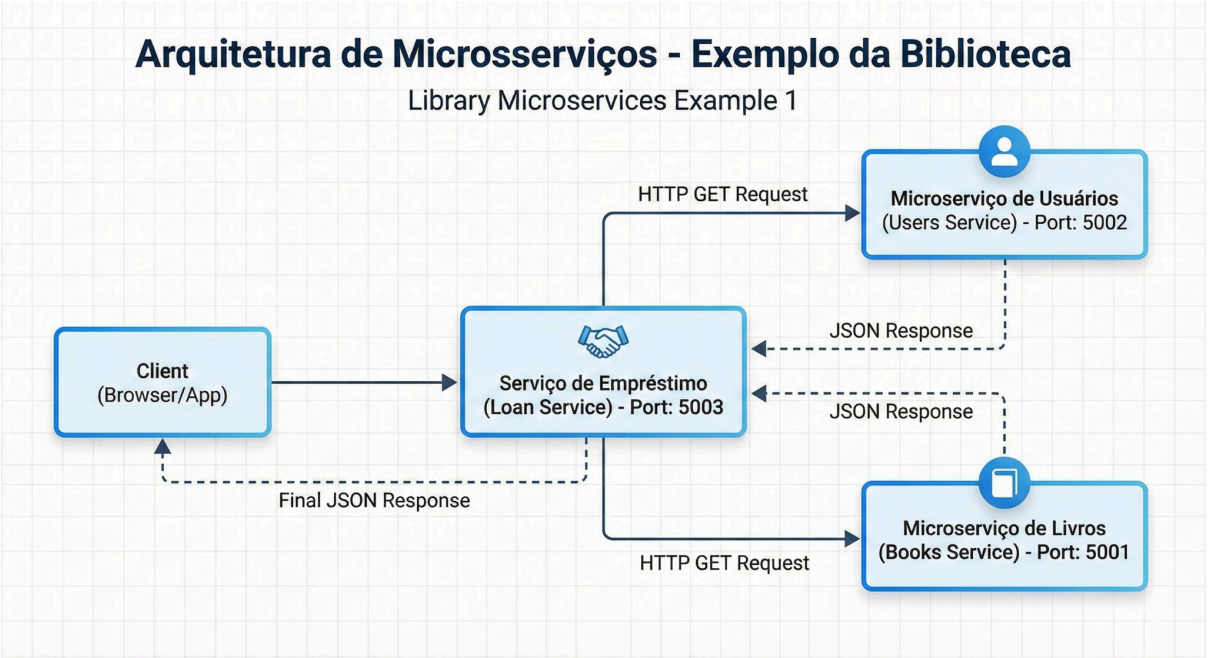
- **Simplicidade:** Permite criar uma API funcional com pouquíssimas linhas de código.
- **Portabilidade:** Cada serviço roda em sua própria porta, simulando perfeitamente servidores independentes.
- **Comunicação:** Integra-se facilmente com a biblioteca requests para chamadas HTTP entre serviços.

*** Lembre-se que você pode utilizar a linguagem de programação que você preferir, irei utilizar o flask por sua simplicidade.**

Vamos começar com um exemplo prático, focado na comunicação entre três serviços de uma biblioteca.

Exemplo 1: Gestão de Empréstimos de livros (Biblioteca)

Na imagem abaixo mostra a arquitetura do sistema de gestão de empréstimo de livros:



1. Este cenário apresenta um sistema de biblioteca modular composto por três **microsserviços independentes** e especializados.
2. Os serviços de **Livros** e **Usuários** atuam como "especialistas" isolados, cada um gerindo sua própria base de dados.
3. O serviço de **Empréstimo** funciona como o orquestrador central, realizando o "meio de campo" via requisições **HTTP/REST**.
4. Essa estrutura garante que o sistema seja **desacoplado**, permitindo que cada parte cresça de maneira independente.

1. Microservice de Livros (Porta 5001)

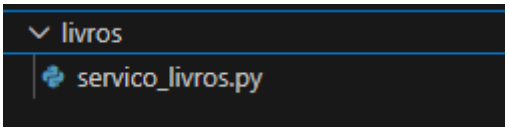
Este código cria um microsserviço em Flask para gerenciar o catálogo de uma biblioteca, fornecendo detalhes de livros em formato JSON através da rota `/livro/<id>` na porta 5001.

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/livro/<id>')
def detalhes_livro(id):
    # Simula base de dados de livros
    catalogo = {"1": "Microservices 101", "2": "Flask for Beginners"}
```

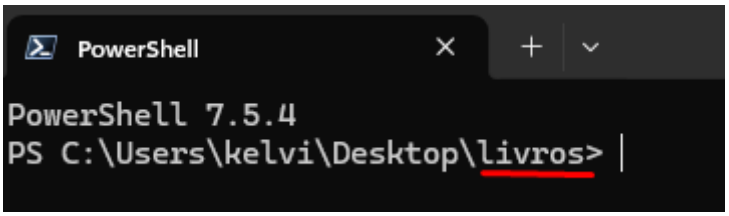
```
    return jsonify({"id": id, "titulo": catalogo.get(id,
"Desconhecido"), "disponivel": True})

if __name__ == "__main__":
    app.run(port=5001)
```

Agora vamos criar o ambiente para rodar o nosso micro serviço de livros, siga esses passos:

1 - Primeiro, crie uma pasta chamada **livros** e dentro dela crie um arquivo chamado **servico_livros.py** e cole o código acima. A estrutura irá ficar assim:



2 - Acesse o terminal e navegue até o diretório `livros`.



3 - Para criar um ambiente virtual em python é simples rode o comando:

py -m venv venv

```
PS C:\Users\kelvi\Desktop\livros> py -m venv venv
```

4 - Após criar o ambiente virtual, vamos ativá-lo, para isso rode o comando:

.\venv\Scripts\activate

```
PS C:\Users\kelvi\Desktop\livros> .\venv\Scripts\activate
```

5 - Após rodar o comando aparecerá um símbolo (env) que indica que o ambiente virtual está ativado.

```
(venv) PS C:\Users\kelvi\Desktop\livros>
```

6 - Pronto, com o ambiente virtual ativado, vamos instalar o flask. Para instalar rode o comando:

pip install flask

```
(venv) PS C:\Users\kelvi\Desktop\livros> pip install flask
Collecting flask
  Downloading flask-3.1.2-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB
```

7 - Após a instalação, se rodarmos o comando "**pip list**" veremos o flask instalado:

pip list

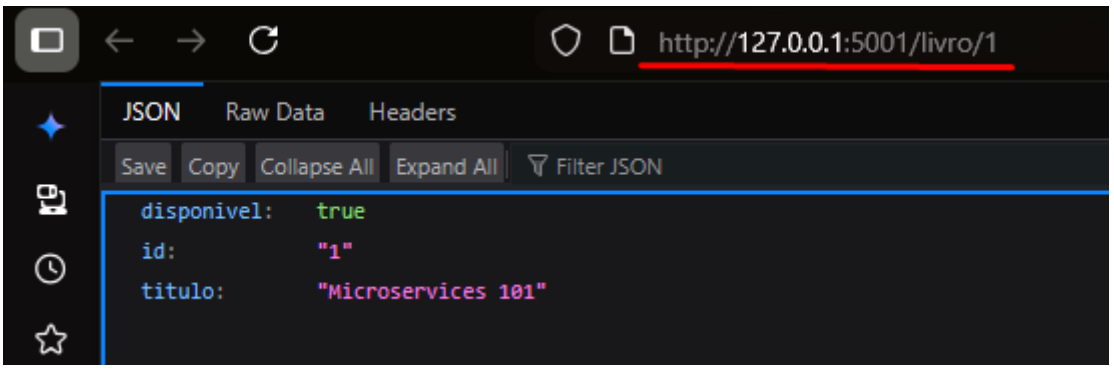
```
(venv) PS C:\Users\kelvi\Desktop\livros> pip list
Package      Version
------------ -------
blinker      1.9.0
click        8.3.1
colorama     0.4.6
Flask        3.1.2
itsdangerous 2.2.0
Jinja2       3.1.6
MarkupSafe   3.0.3
pip          25.3
Werkzeug     3.1.5
```

8 - Estamos prontos para rodar o nosso programa que está no arquivo **servico_livros.py**, para isso rode o comando:

python .\servico_livros.py

```
(venv) PS C:\Users\kelvi\Desktop\livros> python .\servico_livros.py
 * Serving Flask app 'servico_livros'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5001
Press CTRL+C to quit
```

O serviço de livros está pronto, rodando na sua máquina, acessível pela porta 5001. Ao visitar a rota http://127.0.0.1:5001/livro/1 *(essa é a rota que está indicada na imagem acima, se a sua rota for diferente, adapte)*, você receberá o JSON correspondente ao livro de ID 1:



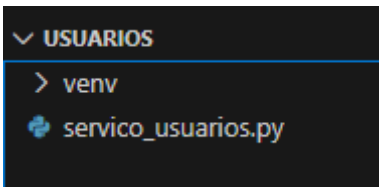
2. Microserviço de Usuários (Porta 5002)

Este código cria um micro serviço que consulta e retorna o status de um usuário em formato JSON através da rota `/usuario/<id>` na porta 5002.

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/usuario/<id>')
def status_usuario(id):
    # Simula verificação de situação do aluno
    return jsonify({"id": id, "nome": "Aluno Exemplo", "status":
"ativo"})

if __name__ == "__main__":
    app.run(port=5002)
```

Para configurar esse ambiente do serviço de Usuários siga os passos:

1 - Crie uma pasta chamada **usuarios** e crie nessa pasta um arquivo chamado **servico_usuarios.py** e cole o código que aparece acima nele.



2 - Acesse o terminal e navegue até o diretório `usuários`.

3 - Crie um ambiente virtual e ative:

py -m venv venv

.\venv\Scripts\activate

4 - instale o flask:

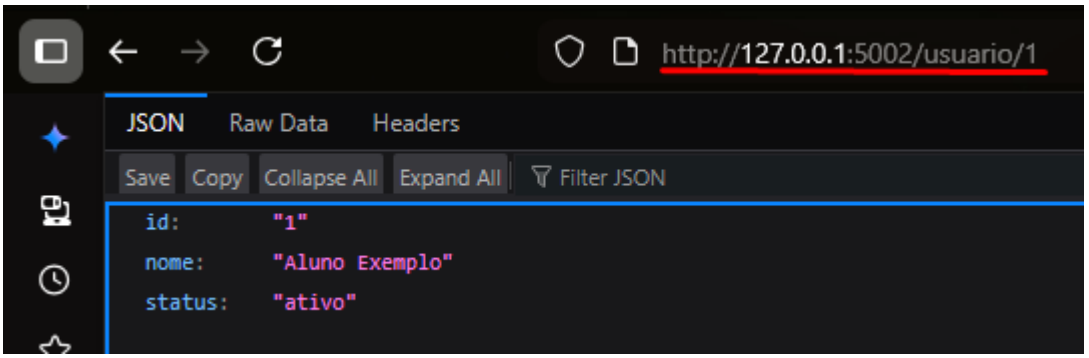
pip install flask

5 - Estamos prontos para rodar o nosso programa que está no arquivo servico_usuarios.py, para isso rode o comando:

python .\servico_usuarios.py

```
(venv) PS C:\Users\kelvi\Desktop\usuarios> python .\servico_usuarios.py
 * Serving Flask app 'servico_usuarios'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI serve
r instead.
 * Running on http://127.0.0.1:5002
Press CTRL+C to quit
```

Ao acessar a rota: http://127.0.0.1:5002/usuario/1 receberemos:



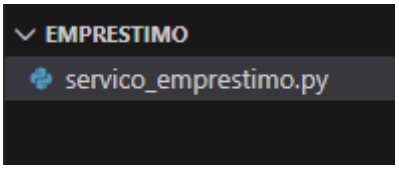
3. Serviço de Empréstimo (Porta 5003) - O Requisitante

Este código implementa um microsserviço de empréstimo orquestrador que consome dados de usuários e livros de outros serviços via HTTP/REST para autorizar transações em formato JSON na porta 5003.

```
import requests
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/solicitar/<user_id>/<book_id>')
def realizar_emprestimo(user_id, book_id):
    # Requisita dados dos outros microserviços (usuario e livro)
    user_data = requests.get(f'http://localhost:5002/usuario/{user_id}').json()
    book_data = requests.get(f'http://localhost:5001/livro/{book_id}').json()

    if user_data['status'] == "ativo" and book_data['disponivel']:
        return jsonify({
            "mensagem": f"Empréstimo do livro '{book_data['titulo']}' autorizado
para {user_data['nome']}!"
        })
    return jsonify({"erro": "Empréstimo negado."}), 400

if __name__ == "__main__":
    app.run(port=5003)
```

Para configurar esse ambiente do serviço de Empréstimo siga os passos:

1 - Crie uma pasta chamada **emprestimo** e crie nessa pasta um arquivo chamado **servico_emprestimo.py** e cole o código que aparece acima nele.



2 - Acesse o terminal e navegue até o diretório **emprestimo**.

3 - Crie um ambiente virtual e ative.

4 - instale o flask e o requests. Para instalar o requests rode o comando:

pip install requests

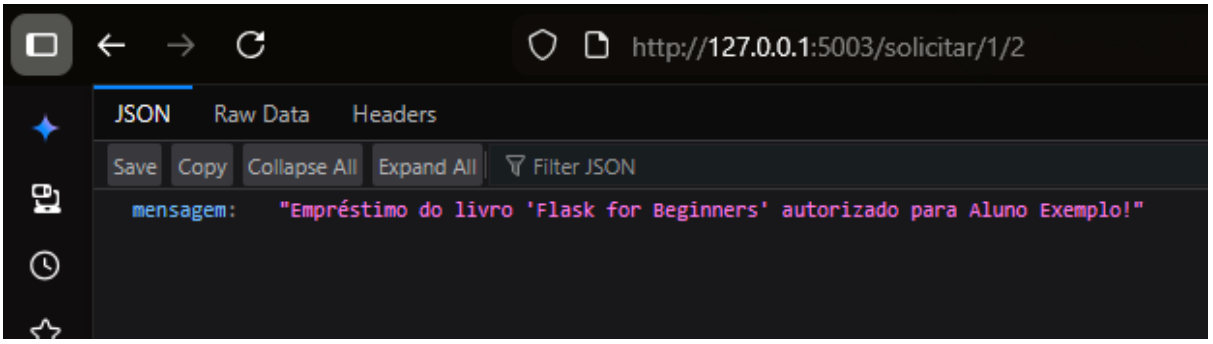
```
(venv) PS C:\Users\kelvi\Desktop\emprestimo> pip install requests
Collecting requests
  Using cached requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Using cached charset_normalizer-3.4.4-cp314-cp314-win_amd64.whl.metadata (38 k
Collecting idna<4,>=2.5 (from requests)
  Using cached idna-3.11-py3-none-any.whl.metadata (8.4 kB)
```

5 - Estamos prontos para rodar o nosso programa que está no arquivo **servico_emprestimo.py**, para isso rode o comando:

python .\servico_emprestimo.py

```
(venv) PS C:\Users\kelvi\Desktop\emprestimo> python .\servico_emprestimo.py
 * Serving Flask app 'servico_emprestimo'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5003
Press CTRL+C to quit
```

Ao acessar a rota: http://127.0.0.1:5003/solicitar/1/2 receberemos:



Desativando os ambientes virtuais:

Após a conclusão dos testes dos serviços, é necessário finalizar os processos em execução e desativar os ambientes virtuais. Para prosseguir, siga as instruções detalhadas a seguir:

1 - Para cada terminal aberto, você aperta o **Ctrl + C** e o serviço será encerrado.

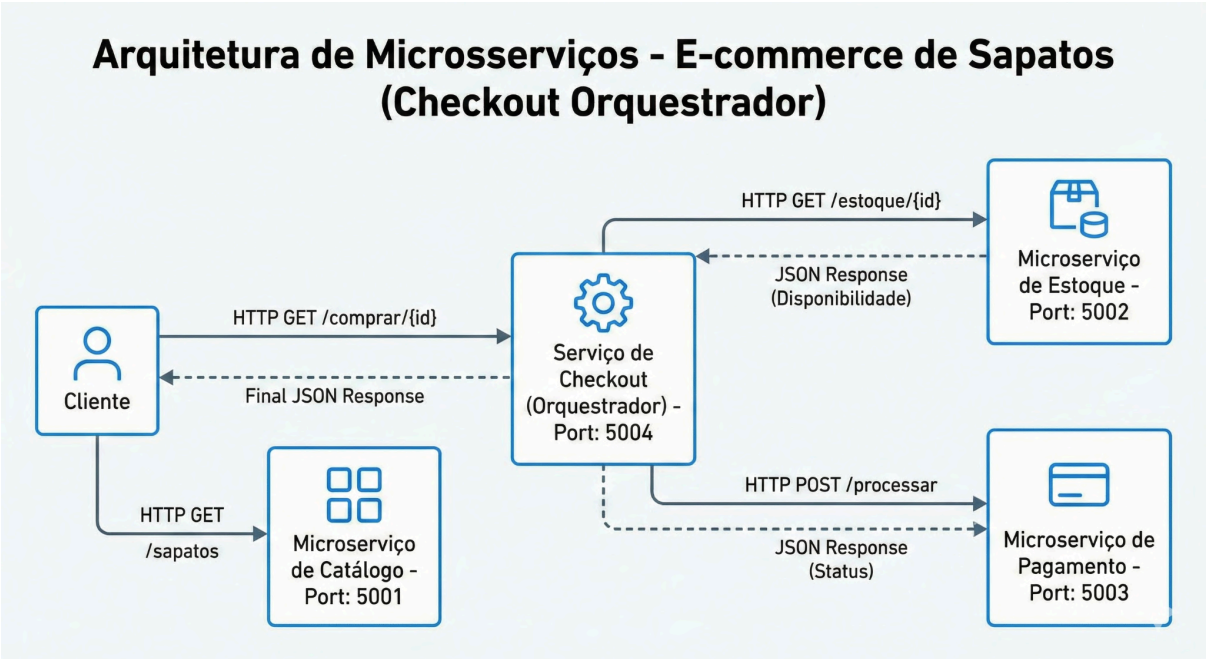
```
 * Serving Flask app 'servico_livros'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a
 * Running on http://127.0.0.1:5001
Press CTRL+C to quit
(venv) PS C:\Users\kelvi\Desktop\livros>
```

2 - Para desativar os ambientes virtuais é só digitar “**deactivate”** e o (venv) será desativado.

```
(venv) PS C:\Users\kelvi\Desktop\livros> deactivate
PS C:\Users\kelvi\Desktop\livros>
```

Ótimo, agora estamos prontos para o próximo exemplo.

Exemplo 2: E-commerce de Sapatos (Arquitetura com 4 Serviços)



1. Este cenário simula um ecossistema de vendas onde cada etapa da compra é gerenciada por um **especialista independente**.
2. Os serviços de **Catálogo**, **Estoque**, **Pagamento e Checkout** rodam em portas separadas, garantindo que o sistema não seja um monstro indivisível.
3. O **Serviço de Checkout** orquestra a venda, "conversando" com os outros três via **HTTP/REST** para validar cada passo.
4. Essa divisão permite que o catálogo de sapatos continue visível mesmo que o processamento de pagamentos sofra uma instabilidade temporária.

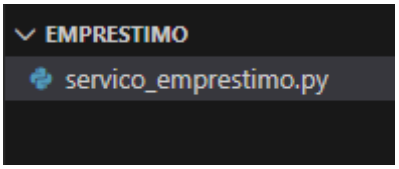
1. Microserviço de Catálogo (Porta 5001)

Este código define um microsserviço de catálogo que lista os modelos de calçados disponíveis em formato JSON através da rota `/sapatos` na porta 5001.

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/sapatos')
def listar_sapatos():
    # Retorna o catálogo dos sapatos.
    return jsonify([
        {"id": "s1", "modelo": "Tênis Runner", "preco": 299.90},
        {"id": "s2", "modelo": "Bota Couro", "preco": 450.00}
    ])

if __name__ == "__main__":
    app.run(port=5001)
```

Para configurar esse ambiente do serviço de Catálogo siga os passos:

1 - Crie uma pasta chamada **catalogo** e crie nessa pasta um arquivo chamado **servico_catalogo.py** e cole o código que aparece acima nele. A pasta irá ficar assim:



2 - Acesse o terminal e navegue até o diretório **catalogo**.

3 - Crie um ambiente virtual e ative.

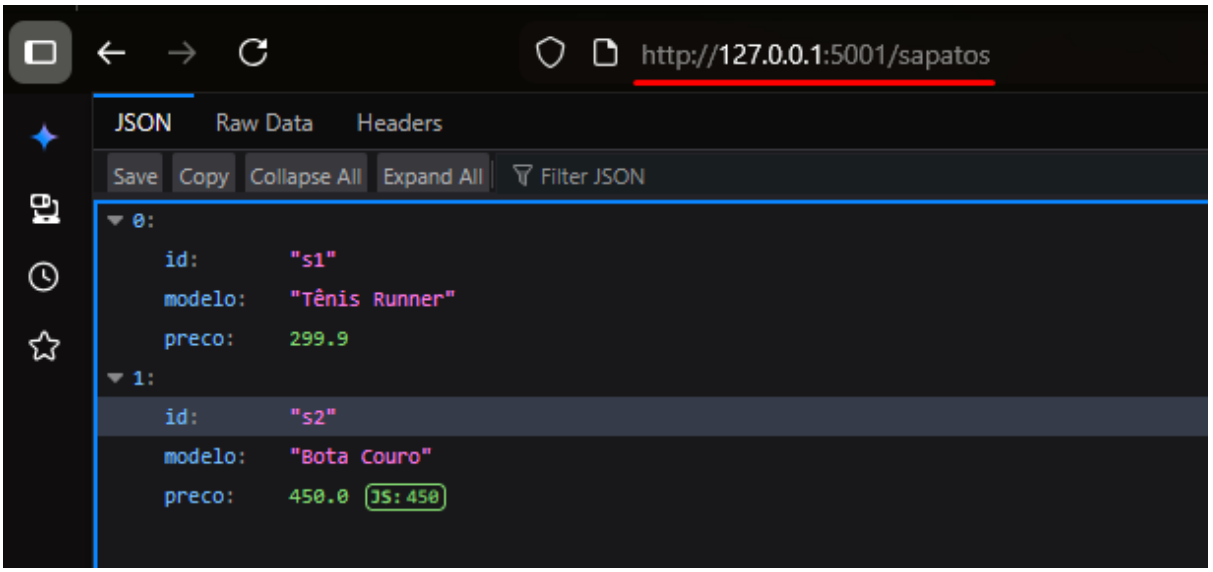
4 - instale o flask.

5 - Estamos prontos para rodar o nosso programa que está no arquivo **servico_catalogo.py**, para isso rode o comando:

python .\servico_catalogo.py

```
(venv) PS C:\Users\kelvi\Desktop\catalogo> python .\servico_catalogo.py
 * Serving Flask app 'servico_catalogo'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI serve
r instead.
 * Running on http://127.0.0.1:5001
Press CTRL+C to quit
```

Ao acessar a rota: http://127.0.0.1:5001/sapatos receberemos:



2. Microserviço de Estoque (Porta 5002)

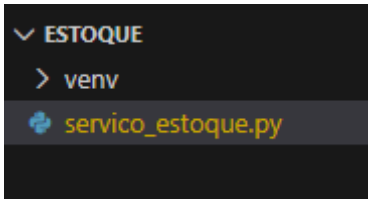
Este código implementa um microsserviço especializado que verifica e retorna a disponibilidade de um produto em formato JSON pela rota `/estoque/<id>` na porta 5002.

```python
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/estoque/<id>')
def checar_estoque(id):
    # Simula se o tamanho solicitado está disponível
    estoque = {"s1": True, "s2": False}
    return jsonify({"id": id, "disponivel": estoque.get(id, False)})

if __name__ == "__main__":
```

```
    app.run(port=5002)
```

Para configurar esse ambiente do serviço de Estoque siga os passos:

1 - Crie uma pasta chamada **estoque** e crie nessa pasta um arquivo chamado **servico_estoque.py** e cole o código que aparece acima nele. A pasta irá ficar assim:



2 - Acesse o terminal e navegue até o diretório **estoque**.

3 - Crie um ambiente virtual e ative.

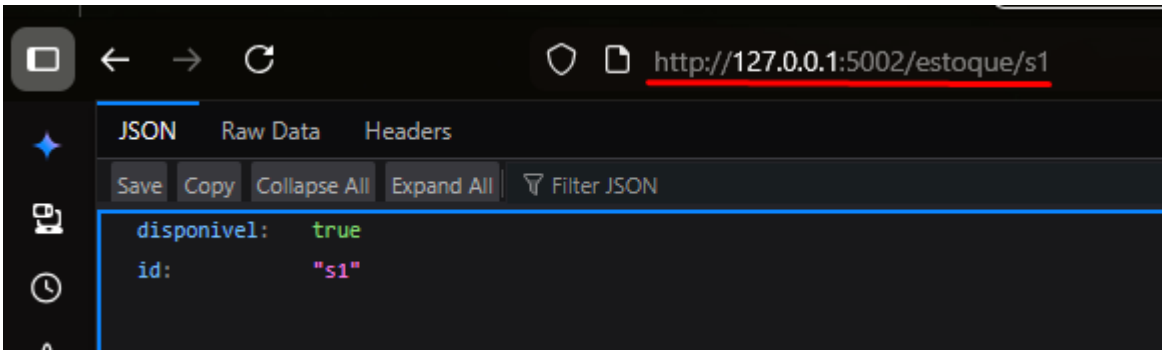
4 - instale o flask.

5 - Estamos prontos para rodar o nosso programa que está no arquivo **servico_estoque.py**, para isso rode o comando:

python .\servico_estoque.py

```
(venv) PS C:\Users\kelvi\Desktop\estoque> python .\servico_estoque.py
 * Serving Flask app 'servico_estoque'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deploy
 * Running on http://127.0.0.1:5002
Press CTRL+C to quit
```

Ao acessar a rota: http://127.0.0.1:5002/estoque/s1 receberemos:



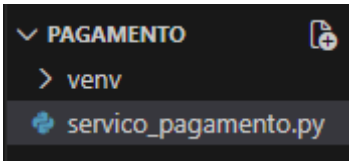
3. Microserviço de Pagamento (Porta 5003)

Este código cria um microsserviço de pagamento em Flask que processa transações financeiras via requisições POST na rota `/processar` (porta 5003), retornando o status e o ID da transação em formato JSON.

```
from flask import Flask, jsonify, request

app = Flask(__name__)

@app.route('/processar', methods=['POST'])
def pagar():
    dados = request.json
    # Simula a aprovação financeira
    return jsonify({"transacao_id": "999", "status": "aprovado",
"valor": dados['total']})

if __name__ == "__main__":
    app.run(port=5003)
```

Para configurar esse ambiente do serviço de Pagamento siga os passos:

1 - Crie uma pasta chamada **pagamento** e crie nessa pasta um arquivo chamado **servico_pagamento.py** e cole o código que aparece acima nele. A pasta irá ficar assim:



2 - Acesse o terminal e navegue até o diretório **pagamento**.

3 - Crie um ambiente virtual e ative.

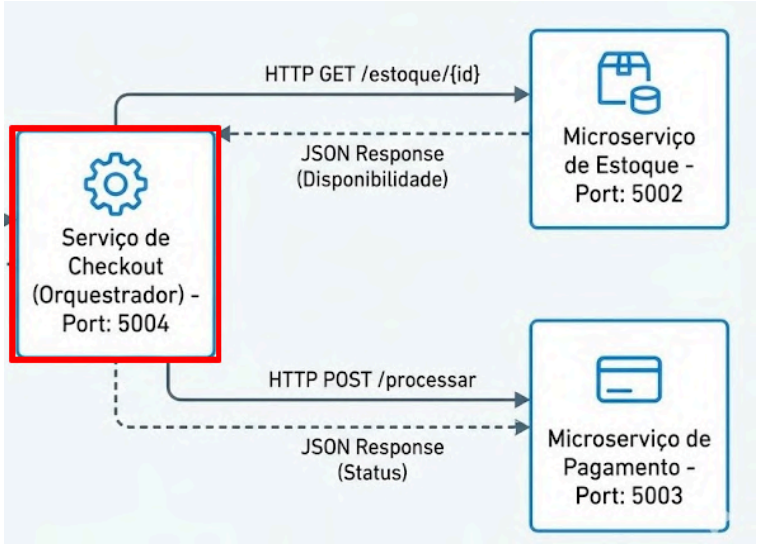
4 - instale o flask.

5 - Estamos prontos para rodar o nosso programa que está no arquivo **servico_pagamento.py**, para isso rode o comando:

python .\servico_pagamento.py

```
(venv) PS C:\Users\kelvi\Desktop\pagamento> python .\servico_pagamento.py
 * Serving Flask app 'servico_pagamento'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deploymen
 * Running on http://127.0.0.1:5003
Press CTRL+C to quit
```

4. Serviço de Checkout (Porta 5004) - O Requisitante



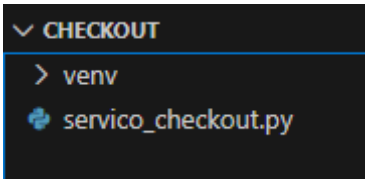
Este código implementa o serviço de checkout orquestrador que coordena a compra, comunicando-se com os microsserviços de estoque e pagamento na porta 5004 para finalizar o pedido em formato JSON.

```
import requests
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/comprar/<id>', methods=['GET'])
def finalizar_pedido(id):
    # 1. Verifica disponibilidade no Estoque (Porta 5002)
    tem_no_estoque = requests.get(f'http://localhost:5002/estoque/{id}').json()

    if not tem_no_estoque['disponivel']:
        return jsonify({"erro": "Sapato fora de estoque"}), 400

    # 2. Processa o Pagamento (Porta 5003) enviando o valor fixo
    pagamento = requests.post('http://localhost:5003/processar', json={"total":
299.90}).json()
```

```
    return jsonify({
        "status": "Pedido Finalizado!",
        "detalhes": pagamento
    })

if __name__ == "__main__":
    app.run(port=5004)
```

Para configurar esse ambiente do serviço de Empréstimo siga os passos:

1 - Crie uma pasta chamada **checkout** e crie nessa pasta um arquivo chamado **servico_checkout.py** e cole o código que aparece acima nele.



2 - Acesse o terminal e navegue até o diretório **checkout**.

3 - Crie um ambiente virtual e ative.

4 - instale o flask e o requests. Para instalar o requests rode o comando:

pip install requests

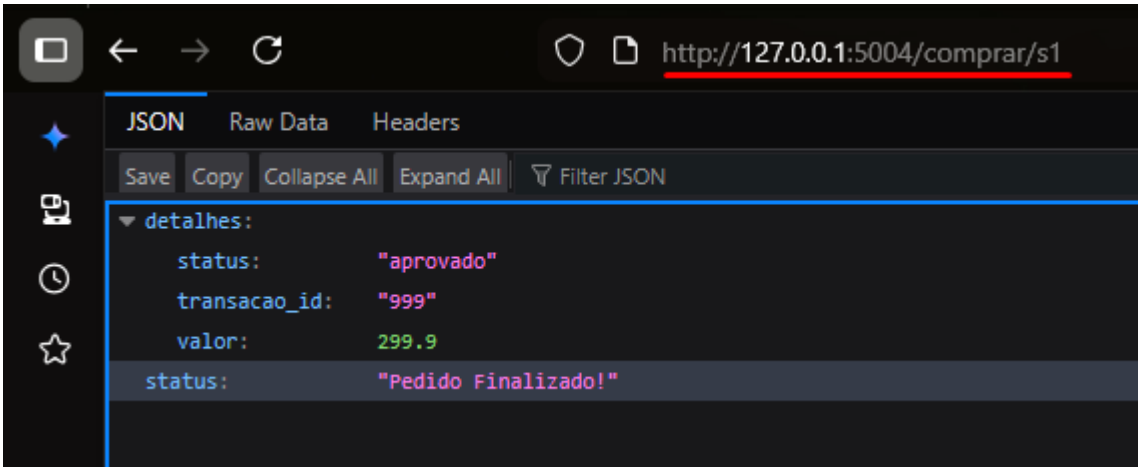
```
(venv) PS C:\Users\kelvi\Desktop\emprestimo> pip install requests
Collecting requests
  Using cached requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Using cached charset_normalizer-3.4.4-cp314-cp314-win_amd64.whl.metadata (38 k
```

5 - Estamos prontos para rodar o nosso programa que está no arquivo **servico_checkout.py**, para isso rode o comando:

python .\servico_checkout.py

```
(venv) PS C:\Users\kelvi\Desktop\checkout> python .\servico_checkout.py
 * Serving Flask app 'servico_checkout'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Us
 * Running on http://127.0.0.1:5004
Press CTRL+C to quit
```

Ao acessar a rota: http://127.0.0.1:5004/comprar/s1 receberemos:


http://127.0.0.1:5004/comprar/s1
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
detalhes:
status: "aprovado"
transacao_id: "999"
valor: 299.9
status: "Pedido Finalizado!"